

# COMPUTER VISION LAB 01 — MIDTERM CHEAT SHEET

Complete Exam Guide for Manual 01: Introduction to Computer Vision & Image Processing | AI-4002 | NUCES Karachi

## PART 1: CORE THEORETICAL FOUNDATIONS

### 1. Introduction to Computer Vision (CV) & Digital Image Processing (DIP)

**Computer Vision (CV):** A multidisciplinary field enabling computers to interpret, analyze, and understand visual data from the physical world, imitating human visual capabilities using algorithmic models for automated decision-making.

**Digital Image Processing (DIP):** The manipulation of digital images using computer algorithms to extract meaningful structural information or enhance visual quality.

**DIP vs. Analog Processing:** Digital processing offers immense advantages over analog methods because digital algorithms prevent signal distortion, physical degradation, and noise accumulation while allowing complex multi-stage filtering.

**Core Aim:** To bridge the operational gap between visual hardware acquisition (cameras/sensors) and high-level artificial intelligence software.

### 2. Complete 25-Key Terminology Dictionary

| Term               | Exam Definition & Core Concept                                                   |
|--------------------|----------------------------------------------------------------------------------|
| Image              | A 2D spatial array of pixel values representing visual information $f(x,y)$ .    |
| Pixel              | Short for 'picture element'; smallest discrete unit carrying intensity/color.    |
| Resolution         | Spatial dimensions of an image, expressed as Width x Height in pixels.           |
| Grayscale          | Single-channel image storing intensity values from 0 (black) to 255 (white).     |
| RGB                | Additive color space combining Red, Green, Blue channels per pixel.              |
| Feature            | A distinctive, recognizable visual structure or pattern in an image.             |
| Feature Extraction | Isolating salient visual features via mathematical kernels.                      |
| Edge Detection     | Locating sharp intensity transitions and spatial boundaries.                     |
| Object Detection   | Locating and identifying object instances (class label + bounding box).          |
| Segmentation       | Partitioning an image into meaningful, distinct structural regions.              |
| Image Processing   | Enhancing raw visual data or extracting low-level features.                      |
| CNN                | Convolutional Neural Network; deep architecture using 2D spatial kernels.        |
| Deep Learning      | Multi-layered neural networks for automatic representation learning.             |
| Feature Map        | 2D spatial output matrix resulting from convolving a filter kernel.              |
| Object Recognition | Identifying and classifying visual instances within media.                       |
| Classification     | Assigning a categorical label to an entire input image.                          |
| Image Registration | Spatially aligning multiple images into a unified coordinate system.             |
| Optical Flow       | Estimating 2D motion vector fields between consecutive video frames.             |
| Tracking           | Following the spatial movement trajectory of objects across frames.              |
| Homography         | A 3x3 transformation matrix relating two 2D planar projections.                  |
| Feature Descriptor | A compact, invariant numerical vector representing local features.               |
| Histogram          | Graphical frequency distribution of pixel intensity levels in an image.          |
| SIFT               | Scale-Invariant Feature Transform; detects scale & rotation invariant keypoints. |
| SURF               | Speeded-Up Robust Features; fast accelerated approximation of SIFT.              |

|     |                                                                                       |
|-----|---------------------------------------------------------------------------------------|
| HOG | Histogram of Oriented Gradients; counts localized gradient orientation distributions. |
|-----|---------------------------------------------------------------------------------------|

3. Real-World Applications

- **Autonomous Vehicles:** Self-driving perception, lane mark tracking, traffic sign recognition, pedestrian detection, obstacle avoidance.
- **Medical Imaging:** Automated diagnosis, anomaly segmentation in X-rays, MRIs, CT scans, and ultrasound imaging.
- **Robotics:** Visual servoing, spatial navigation, robotic object grasping, automated industrial quality inspection.
- **Object Detection & Recognition:** Security surveillance, automated retail checkout, inventory tracking.
- **Facial Recognition:** Biometric authentication, security access control, automated smartphone unlocking.
- **AR/VR:** Spatial tracking, overlaying 3D digital assets onto live camera streams.

4. Purpose of Image Processing (5 Core Groups)

1. **Visualization:** Revealing objects/structures not visible to the human eye.
2. **Image Sharpening & Restoration:** Improving resolution and removing blur/noise.
3. **Image Retrieval:** Searching and browsing relevant images from large digital archives.
4. **Measurement of Pattern:** Measuring physical/spatial dimensions of objects within an image.
5. **Image Recognition:** Distinguishing, categorizing, and labeling individual objects.

5. Fundamental Steps of Digital Image Processing (11 Stages)

The DIP workflow consists of 11 sequential stages interacting with a central **Knowledge Base**:

| DIP Stage                       | Core Purpose & Exam Detail                                                            |
|---------------------------------|---------------------------------------------------------------------------------------|
| 1. Image Acquisition            | First stage; captures raw digital image and applies basic pre-processing (scaling).   |
| 2. Image Enhancement            | Manipulates contrast/brightness to make hidden features more prominent.               |
| 3. Image Restoration            | Reverses modeled degradation, noise, and optical blur mathematically.                 |
| 4. Color Image Processing       | Handles digital color spaces (RGB, HSV, BGR) and color manipulation.                  |
| 5. Wavelets & Multi-Res         | Represents images at varying scale resolutions for pyramidal analysis.                |
| 6. Compression                  | Reduces file size and bandwidth requirements for data storage/transmission.           |
| 7. Morphological Processing     | Extracts structural shape components useful in region boundary analysis.              |
| 8. Segmentation                 | Partitions an image into constituent parts; noted as ONE OF THE MOST DIFFICULT TASKS. |
| 9. Representation & Description | Transforms raw segmented pixels into boundary/region descriptors for computers.       |
| 10. Object Recognition          | Assigns structural categorical labels (e.g., 'car') based on computed descriptors.    |
| 11. Knowledge Base              | Central repository guiding and limiting search processes at every DIP stage.          |

MCQ TRAPS & EXAM TRICK ALERT: DIP Pipeline Output Classification Rule

- **Steps 1 to 8:** The output of these processing steps is generally **AN IMAGE**.
- **Steps 9 to 10:** The output transitions from images to **IMAGE ATTRIBUTES** (labels, vectors, descriptors).
- **Hardest Step:** Manual 01 explicitly singles out **Segmentation** as one of the most difficult tasks in DIP.

PART 2: ESSENTIAL COMPUTER VISION LIBRARIES MATRIX

| Library      | Main Focus             | Base        | Language    | Abstraction | Community | DL Integration         |
|--------------|------------------------|-------------|-------------|-------------|-----------|------------------------|
| OpenCV       | General CV & IP        | Independent | C++, Python | High        | Strong    | Python bindings        |
| Scikit-Image | Image Processing       | Independent | Python      | Medium      | Moderate  | NumPy / Matplotlib     |
| Pillow       | Basic IP / I/O         | Independent | Python      | Low         | Moderate  | Basic array conversion |
| TorchVision  | CV for PyTorch         | PyTorch     | Python      | High        | Strong    | Native PyTorch         |
| TensorFlow   | General ML / CV        | TensorFlow  | Python      | High        | Strong    | Native TensorFlow      |
| Keras        | High-level Neural Nets | TF / Theano | Python      | High        | Strong    | Native High-Level API  |

|              |                     |            |             |        |          |                       |
|--------------|---------------------|------------|-------------|--------|----------|-----------------------|
| OpenVINO     | Inference Opt.      | Intel      | C++, Python | High   | Moderate | Framework conversion  |
| PyTorch      | General ML / CV     | PyTorch    | Python      | High   | Strong   | Native PyTorch        |
| Hugging Face | Pretrained Models   | PyTorch/TF | Python      | High   | Strong   | Transformers Hub      |
| Caffe        | Deep Learning / CNN | Caffe      | C++         | Medium | Moderate | C++ CNN Focus         |
| Detectron2   | Object Detection    | PyTorch    | Python      | High   | Strong   | Specialized Detection |

PART 3: DIGITAL IMAGE MATHEMATICS & COORDINATE SYSTEM

An image is represented mathematically as a 2D spatial intensity function  $f(x, y)$  where  $x$  and  $y$  are spatial plane coordinates, and  $f$  is the light intensity level at that pixel coordinate.

Coordinate Convention Rules:

- **Origin (0,0):** Located strictly at the **TOP-LEFT CORNER** of the digital image.
- **x-Axis (Rows / Height):** Runs **vertically downwards** from 0 to M-1 (where M = total rows).
- **y-Axis (Columns / Width):** Runs **horizontally to the right** from 0 to N-1 (where N = total columns).

PART 4: PRACTICAL CODE EXAMPLES & PARAMETER BREAKDOWN

1. Reading & Displaying Images

PYTHON EXAM CODE: Reading & Displaying Images

```
import cv2
import matplotlib.pyplot as plt

# 1. Read image from disk into a NumPy ndarray
# Default color space loaded by OpenCV is BGR (Blue, Green, Red)
image_bgr = cv2.imread('Sukuna.jpeg')

# 2. Convert BGR to RGB so Matplotlib displays true colors
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# 3. Render using Matplotlib
plt.imshow(image_rgb)
plt.title('Original Image (RGB)')
plt.axis('off') # Hide coordinate axis ticks
plt.show()
```

| Parameter    | Type / Range              | Exam Meaning & Purpose                                                       |
|--------------|---------------------------|------------------------------------------------------------------------------|
| cv2.imread   | filename: str, flags: int | Loads image from path. Default flag = cv2.IMREAD_COLOR. Returns BGR array.   |
| cv2.cvtColor | src: ndarray, code: int   | Converts color space. cv2.COLOR_BGR2RGB fixes Matplotlib blue color swap.    |
| plt.imshow   | X: array-like, cmap: str  | Displays 2D/3D image arrays. Requires cmap='gray' for single-channel images. |

2. Grayscale Conversion

PYTHON EXAM CODE: Grayscale Conversion

```
import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')

# Convert 3-channel BGR image (H, W, 3) to 1-channel Grayscale (H, W)
gray_image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# Display single-channel grayscale image using cmap='gray'
plt.imshow(gray_image, cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')
plt.show()
```

| Parameter          | Type / Range          | Exam Meaning & Purpose                                                                 |
|--------------------|-----------------------|----------------------------------------------------------------------------------------|
| cv2.COLOR_BGR2GRAY | Color Conversion Flag | Reduces 3-channel BGR into 1-channel intensity using $Y = 0.299R + 0.587G + 0.114B$ .  |
| cmap='gray'        | Matplotlib Colormap   | MANDATORY parameter for 2D single-channel arrays; otherwise displays in greenish tint. |

### 3. Resizing Images

#### PYTHON EXAM CODE: Resizing Images

```
import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Define explicit target dimensions: Width = 300, Height = 200
target_width = 300
target_height = 200

# Resize image (CRITICAL TRAP: dsize takes (width, height))
resized_image = cv2.resize(image_rgb, (target_width, target_height))

# Alternative: Resize by scale factors fx, fy
# resized_scaled = cv2.resize(image_rgb, (0, 0), fx=0.5, fy=0.5)

plt.imshow(resized_image)
plt.title('Resized Image (300x200)')
plt.axis('off')
plt.show()
```

| Parameter  | Type / Range               | Exam Meaning & Purpose                                                         |
|------------|----------------------------|--------------------------------------------------------------------------------|
| cv2.resize | src, dsize, fx, fy, interp | Resizes image array. dsize tuple MUST be (width, height), NOT (height, width)! |
| dsize      | tuple (width, height)      | Target dimensions in pixels. Set to (0,0) if using fx and fy scaling factors.  |
| fx, fy     | float > 0.0                | Scale factors along horizontal (fx) and vertical (fy) axes.                    |

### 4. Image Filtering & Blurring

#### PYTHON EXAM CODE: Image Blurring

```
import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Apply Normalized Box Filter (Averaging Blur) with 5x5 kernel
blurred_image = cv2.blur(image_rgb, (5, 5))

# Alternative: Gaussian Blur with standard deviation sigmaX = 0
# blurred_gaussian = cv2.GaussianBlur(image_rgb, (5, 5), 0)

plt.imshow(blurred_image)
plt.title('Blurred Image (5x5 Kernel)')
plt.axis('off')
plt.show()
```

| Parameter | Type / Range          | Exam Meaning & Purpose                                                            |
|-----------|-----------------------|-----------------------------------------------------------------------------------|
| cv2.blur  | src, ksize=(w, h)     | Smooths image by convolving with a normalized box filter of width w and height h. |
| ksize     | tuple (width, height) | Kernel window size. Must be positive odd integers (e.g., (3,3), (5,5), (7,7)).    |

## 5. Image Cropping via NumPy Array Slicing

### PYTHON EXAM CODE: Image Cropping

```
import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Get spatial height and width from array shape
height, width = image_rgb.shape[:2]

# Compute horizontal midpoint integer
mid_x = width // 2

# Crop left half using NumPy slicing: [y_start:y_end, x_start:x_end]
# Vertical rows (y) come FIRST, horizontal columns (x) come SECOND
cropped_left = image_rgb[:, :mid_x]

plt.imshow(cropped_left)
plt.title('Cropped Image (Left Half)')
plt.axis('off')
plt.show()
```

| Parameter           | Type / Range            | Exam Meaning & Purpose                                                     |
|---------------------|-------------------------|----------------------------------------------------------------------------|
| image[y1:y2, x1:x2] | NumPy Array Slicing     | OpenCV HAS NO cv2.crop()! Cropping is performed using NumPy array slicing. |
| y1:y2               | Vertical Row Range      | Selects vertical row span from top index y1 to bottom index y2.            |
| x1:x2               | Horizontal Column Range | Selects horizontal column span from left index x1 to right index x2.       |

## 6. Adding Text Annotations

### PYTHON EXAM CODE: Adding Text

```

import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Annotate text on image
# Parameters: img, text, org(x,y), fontFace, fontScale, color, thickness
cv2.putText(
    image_rgb,
    'Sukuna CV Lab',
    (50, 100),           # Point (x=50, y=100) -> 50px right, 100px down
    cv2.FONT_HERSHEY_SIMPLEX, # Font type
    1.0,                # Font scale factor
    (255, 0, 0),         # Red color in RGB space
    2,                  # Line thickness in pixels
)

plt.imshow(image_rgb)
plt.axis('off')
plt.show()

```

| Parameter   | Type / Range                                  | Exam Meaning & Purpose                                      |
|-------------|-----------------------------------------------|-------------------------------------------------------------|
| cv2.putText | img, text, org, font, scale, color, thickness | Draws text string onto image array in-place.                |
| org         | tuple (x, y)                                  | Bottom-left origin coordinate of the text string in pixels. |
| color       | tuple (B,G,R) or (R,G,B)                      | Text color tuple matching the image array color space.      |

## 7. Analyzing Image Data using Pandas

### PYTHON EXAM CODE: Pandas Analysis

```

import cv2
import pandas as pd

image_bgr = cv2.imread('Sukuna.jpeg')
gray_image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# Convert 2D grayscale intensity matrix into a Pandas DataFrame
df = pd.DataFrame(gray_image)

# Summary statistics of pixel intensities column by column
print("Shape of DataFrame:", df.shape)
print("Pixel Statistics:\n", df.describe())

```

| Parameter          | Type / Range        | Exam Meaning & Purpose                                                             |
|--------------------|---------------------|------------------------------------------------------------------------------------|
| pd.DataFrame(gray) | 2D Array Conversion | Maps 2D image matrix where rows = image height and columns = image width.          |
| df.describe()      | Tabular Statistics  | Generates summary count, mean, std, min, 25%, 50%, 75%, max for pixel intensities. |

## 8. Image Thresholding (Binary)

### PYTHON EXAM CODE: Binary Thresholding

```

import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
gray_image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# Apply Binary Thresholding
# Syntax: cv2.threshold(src, thresh, maxval, type)
ret, thresholded_image = cv2.threshold(gray_image, 100, 200, cv2.THRESH_BINARY)

plt.imshow(thresholded_image, cmap='gray')
plt.title('Thresholded Image (Binary)')
plt.axis('off')
plt.show()

```

| Parameter         | Type / Range              | Exam Meaning & Purpose                                                           |
|-------------------|---------------------------|----------------------------------------------------------------------------------|
| cv2.threshold     | src, thresh, maxval, type | Converts single-channel grayscale image to binary. RETURNS TUPLE (ret, dst_img)! |
| thresh            | float / int (e.g., 100)   | Intensity limit threshold value.                                                 |
| maxval            | float / int (e.g., 200)   | Maximum value assigned to pixels exceeding the threshold limit.                  |
| cv2.THRESH_BINARY | Thresholding Flag         | Equation: $dst(x,y) = maxval$ if $src(x,y) > thresh$ else 0.                     |

## 9. Image Rotation

### PYTHON EXAM CODE: Image Rotation

```

import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

height, width = image_rgb.shape[:2]
center = (width // 2, height // 2)

# Compute 2D Rotation Matrix M (Center, Angle in degrees, Scale)
# Angle = +45 degrees counter-clockwise
M = cv2.getRotationMatrix2D(center, 45, 1.0)

# Apply 2D Affine Transformation
rotated_image = cv2.warpAffine(image_rgb, M, (width, height))

plt.imshow(rotated_image)
plt.title('Rotated Image (45 deg)')
plt.axis('off')
plt.show()

```

| Parameter               | Type / Range         | Exam Meaning & Purpose                                                       |
|-------------------------|----------------------|------------------------------------------------------------------------------|
| cv2.getRotationMatrix2D | center, angle, scale | Computes 2x3 affine matrix for rotation. Positive angle = counter-clockwise. |
| cv2.warpAffine          | src, M, dsize        | Applies 2x3 transformation matrix M to image. dsize MUST be (width, height). |

## 10. Image Blending (Addition & Resizing)

### PYTHON EXAM CODE: Image Blending

```

import cv2
import matplotlib.pyplot as plt

# 1. Load two images
img1 = cv2.imread('Sukuna.jpeg')
img2 = cv2.imread('Gojo.jpeg')

img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

# 2. Match dimensions: Resize img1 to match img2 dimensions
target_height, target_width = img2_rgb.shape[:2]
img1_resized = cv2.resize(img1_rgb, (target_width, target_height))

# 3. Blend images using Saturation Arithmetic (cv2.add)
blended_image = cv2.add(img1_resized, img2_rgb)

plt.imshow(blended_image)
plt.title('Blended Image (cv2.add)')
plt.axis('off')
plt.show()

```

| Parameter                   | Type / Range            | Exam Meaning & Purpose                                                                                                               |
|-----------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| cv2.add                     | src1, src2              | Performs element-wise saturation addition: $\text{dst}(x,y) = \min(\text{src1} + \text{src2}, 255)$ . Prevents overflow wrap-around. |
| Dimension Match Requirement | Identical Shape (H,W,C) | Both input matrices MUST have exact same shape; otherwise cv2.error is raised.                                                       |

## 11. Histogram Equalization

### PYTHON EXAM CODE: Histogram Equalization

```

import cv2
import matplotlib.pyplot as plt

image_bgr = cv2.imread('Sukuna.jpeg')
gray_image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

# Equalize histogram to enhance global contrast
equalized_image = cv2.equalizeHist(gray_image)

plt.imshow(equalized_image, cmap='gray')
plt.title('Histogram Equalized Image')
plt.axis('off')
plt.show()

```

| Parameter        | Type / Range              | Exam Meaning & Purpose                                                        |
|------------------|---------------------------|-------------------------------------------------------------------------------|
| cv2.equalizeHist | src: 8-bit single-channel | Stretches intensity distribution across full 0-255 range to enhance contrast. |

## PART 5: QUICK REFERENCE SHEETS & EXAM SURVIVAL KIT

### 1. Master Concepts Summary

- Origin:** Top-Left corner is strictly (0,0).
- Axis Convention:** x = rows (vertically down), y = columns (horizontally right).
- Color Model:** OpenCV loads BGR by default. Convert to RGB before plotting with Matplotlib.
- Resize Shape Order:** image.shape gives (height, width), but cv2.resize takes (width, height).
- Cropping:** No cv2.crop() exists! Use NumPy slicing: image[y1:y2, x1:x2].
- Thresholding Return:** cv2.threshold returns a TUPLE (ret, binary\_image).
- Addition Rule:** cv2.add uses saturation arithmetic (clamped at 255). NumPy + uses modulo (wraps around).

8. **DIP Output Split:** Steps 1-8 output IMAGES; Steps 9-10 output IMAGE ATTRIBUTES.

## 2. Master Functions & APIs Matrix

| Function                | Purpose            | Syntax                                        | Key Parameters                | Return Output       |
|-------------------------|--------------------|-----------------------------------------------|-------------------------------|---------------------|
| cv2.imread              | Read image         | cv2.imread(path)                              | path (str)                    | BGR ndarray         |
| cv2.cvtColor            | Color convert      | cv2.cvtColor(src, code)                       | code (e.g. COLOR_BGR2RGB)     | Converted ndarray   |
| cv2.resize              | Resize array       | cv2.resize(src, dsize)                        | dsize=(width, height)         | Resized ndarray     |
| cv2.blur                | Box filter blur    | cv2.blur(src, ksize)                          | ksize=(w, h) odd tuple        | Smoothed ndarray    |
| cv2.threshold           | Binary thresh      | cv2.threshold(src, th, max, type)             | th, maxval, THRESH_BINARY     | Tuple (ret, dst)    |
| cv2.add                 | Saturated addition | cv2.add(src1, src2)                           | src1, src2 (matching shape)   | Blended ndarray     |
| cv2.putText             | Draw text          | cv2.putText(img, text, org, ...)              | org=(x,y), font, scale, color | Modified image      |
| cv2.getRotationMatrix2D | Compute rotation   | cv2.getRotationMatrix2D(center, angle, scale) | center=(x,y), angle, scale    | 2x3 Affine Matrix   |
| cv2.warpAffine          | Apply rotation     | cv2.warpAffine(src, M, dsize)                 | M, dsize=(width, height)      | Transformed ndarray |
| cv2.equalizeHist        | Enhance contrast   | cv2.equalizeHist(src)                         | src (1-channel grayscale)     | Equalized ndarray   |

## 3. Master Reusable Exam Code Template

### PYTHON EXAM CODE: Master Reusable Exam Boilerplate

```
import cv2
import matplotlib.pyplot as plt

def process_image_pipeline(image_path):
    # 1. Read Image
    img_bgr = cv2.imread(image_path)
    if img_bgr is None:
        raise FileNotFoundError(f"Image not found at {image_path}")

    # 2. Color Conversions
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # 3. Processing Operations
    ret, binary_img = cv2.threshold(img_gray, 128, 255, cv2.THRESH_BINARY)
    blurred_img = cv2.blur(img_rgb, (5, 5))

    # 4. Multi-Plot Rendering
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    axes[0].imshow(img_rgb); axes[0].set_title('Original RGB'); axes[0].axis('off')
    axes[1].imshow(binary_img, cmap='gray'); axes[1].set_title('Binary Threshold'); axes[1].axis('off')
    axes[2].imshow(blurred_img); axes[2].set_title('Blurred (5x5)'); axes[2].axis('off')

    plt.tight_layout()
    plt.show()

# Run Pipeline
# process_image_pipeline('input.jpg')
```

## 4. Formula Sheet

| Formula Name                 | Mathematical / Code Equation                                          |
|------------------------------|-----------------------------------------------------------------------|
| Binary Thresholding Equation | $\text{dst}(x,y) = \text{maxval if src}(x,y) > \text{thresh else } 0$ |

|                                |                                                         |
|--------------------------------|---------------------------------------------------------|
| OpenCV Saturation Addition     | <code>dst(x,y) = min(src1(x,y) + src2(x,y), 255)</code> |
| NumPy Modulo Addition (Avoid!) | <code>dst(x,y) = (src1(x,y) + src2(x,y)) mod 256</code> |
| Horizontal Midpoint Slicing    | <code>mid_x = width // 2</code>                         |
| Grayscale Intensity Conversion | <code>Y = 0.299*R + 0.587*G + 0.114*B</code>            |

5. Easy-to-Confuse Comparisons

Comparison 1: cv2.add() vs. NumPy '+' Operator

| Feature             | cv2.add(A, B)                    | NumPy A + B                               |
|---------------------|----------------------------------|-------------------------------------------|
| Arithmetic Type     | Saturation Arithmetic            | Modulo (Wrap-around) Arithmetic           |
| Overflow Result     | 200 + 100 -> 255 (Clamped White) | 200 + 100 -> 44 (300 mod 256)             |
| Visual Result       | Clean, brightened output         | Severe dark patches & inversion artifacts |
| Exam Recommendation | Mandatory for image blending     | Do not use for uint8 pixel math           |

Comparison 2: Local Descriptors (SIFT vs. SURF vs. HOG)

| Descriptor | Full Name                         | Key Property                           | Primary Focus                 |
|------------|-----------------------------------|----------------------------------------|-------------------------------|
| SIFT       | Scale-Invariant Feature Transform | Invariant to scale and rotation        | Local keypoint detection      |
| SURF       | Speeded-Up Robust Features        | Accelerated box-filter approximation   | Fast local keypoint detection |
| HOG        | Histogram of Oriented Gradients   | Counts localized gradient orientations | Object & pedestrian detection |

MCQ TRAPS & EXAM TRICK ALERT: High-Probability MCQ Facts & Exam Traps

- **Origin Location:** Top-Left corner is strictly (0,0).
- **Axis Orientation:** x is vertical rows (down), y is horizontal columns (right).
- **Color Space Default:** OpenCV loads BGR, Matplotlib expects RGB.
- **Resize Dimension Order:** cv2.resize takes (width, height), NOT (height, width).
- **Cropping Function:** There is NO cv2.crop()! Use NumPy array slicing `image[y1:y2, x1:x2]`.
- **Threshold Return Value:** cv2.threshold returns TWO values: (ret, threshold\_image).
- **Matplotlib Single Channel:** Without `cmap='gray'`, grayscale images plot in greenish colormap.
- **DIP Hardest Task:** Manual 01 designates Segmentation as one of the hardest DIP tasks.
- **Pipeline Transition:** Steps 1-8 output IMAGES; Steps 9-10 output IMAGE ATTRIBUTES.

7. 'Which Function Do I Use?' Decision Matrix

| Task Requirement                               | Appropriate Function / Syntax                                                  |
|------------------------------------------------|--------------------------------------------------------------------------------|
| Load image from disk file                      | <code>cv2.imread('path.jpg')</code>                                            |
| Convert BGR image to RGB for plotting          | <code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>                              |
| Convert BGR image to Grayscale                 | <code>cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)</code>                             |
| Crop Region of Interest (ROI)                  | NumPy Slicing: <code>img[y_start:y_end, x_start:x_end]</code>                  |
| Resize image to specific width & height        | <code>cv2.resize(img, (width, height))</code>                                  |
| Convert grayscale image to black/white binary  | <code>ret, dst = cv2.threshold(gray, thresh, maxval, cv2.THRESH_BINARY)</code> |
| Blend two images safely without overflow       | <code>cv2.add(img1_resized, img2)</code>                                       |
| Render 1-channel image correctly in Matplotlib | <code>plt.imshow(img, cmap='gray')</code>                                      |
| Draw text annotation on image array            | <code>cv2.putText(img, text, (x,y), font, scale, color, thick)</code>          |
| Equalize global contrast                       | <code>cv2.equalizeHist(gray_img)</code>                                        |

8. Common Coding Errors & Fixes

| Problem Scenario | Root Cause | Correct Solution Code |
|------------------|------------|-----------------------|
|------------------|------------|-----------------------|

|                                                       |                                                                                  |                                                                                    |
|-------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Image looks blue/discolored in Matplotlib             | Displaying raw BGR array directly in Matplotlib                                  | <code>img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)</code>                    |
| <code>cv2.error</code> during <code>cv2.add()</code>  | Adding two matrices with different dimensions                                    | Resize <code>img1</code> first: <code>img1_res = cv2.resize(img1, (w2, h2))</code> |
| <code>TypeError</code> during thresholding assignment | Assigning <code>cv2.threshold</code> result to single variable                   | Unpack tuple: <code>ret, thresh_img = cv2.threshold(...)</code>                    |
| Cropped ROI is rotated / cut wrong                    | Confusing NumPy row/col order <code>[y,x]</code> with spatial <code>(x,y)</code> | Remember rows (y) come first: <code>img[y1:y2, x1:x2]</code>                       |
| Grayscale image renders in green/yellow               | Missing <code>cmap='gray'</code> in Matplotlib <code>imshow</code> call          | Include <code>cmap</code> : <code>plt.imshow(gray_img, cmap='gray')</code>         |

## 9. Last-Minute Exam Revision Checklist

Top-Left is (0,0). x = rows (down), y = columns (right).  
 OpenCV loads BGR. Convert to RGB before Matplotlib plotting.  
 Crop using NumPy array slicing: `image[y1:y2, x1:x2]`.  
`cv2.resize` `dsize` parameter is (width, height).  
`cv2.threshold` returns a tuple (ret, binary\_image).  
`cv2.add` uses saturation arithmetic (clamped at 255).  
 Segmentation is one of the most difficult tasks in DIP.  
 Steps 1-8 produce images; Steps 9-10 produce image attributes.